

Progetto 1 – Alternativo

Cattaneo Francesco (matricola 900411), Carpio Herreros Marco (matricola 899802)

Link alla repository: [Repository Metodi](#)

Introduzione

In questo progetto abbiamo implementato una libreria in java che esegue il metodo di Jacobi, di Gauss-Seidel, del Gradiente e del Gradiente Coniugato.

È stata usata la libreria EJML come supporto per lo sviluppo del progetto. In particolare, l'utilizzo di esso si è concentrato sulle strutture dati vettoriali e matriciali, e sulle operazioni tra esse.

L'implementazione dei quattro metodi iterativi e vari metodi di supporto (creazione della matrice triangolare inferiore, controllo dell'esistenza di zeri sulla diagonale, risoluzione del sistema usando la formula forward-substitution) sono stati scritti autonomamente evitando di risolvere i sistemi dei quattro metodi attraverso l'utilizzo di EJML che forniva funzioni per risolvere tali sistemi.

Inoltre, sono stati usati strumenti IA per la risoluzione dei problemi quando necessario.

Come specificato nella traccia del progetto la nostra implementazione si limita al caso di matrice simmetriche e definite positive. Dato che il caso si limita a questa condizione li esperimenti sono concentrati su quattro matrici sparse in formato .mtx, fornite dal prof, con vettore soluzione esatta $x = 1$, membro destro $b = Ax$ e tolleranze 10^{-4} , 10^{-6} , 10^{-8} , 10^{-10} .

Struttura della libreria

Il progetto fa uso di Maven per un controllo delle dipendenze più semplice, la standardizzazione della struttura del progetto e un'automazione più veloce del processo di build. Il controllo delle dipendenze è definito all'interno del file `pom.xml` nella root della repository.

Tutte le classi fanno parte del package `com`, suddiviso in:

- Funzioni: contenente le funzioni ed eventuali strutture dati di supporto
- Metodi: contenente i quattro metodi iterativi

Inoltre, è presente una cartella contenente le quattro matrici (`spa1`, `spa2`, `vem1`, `vem2`) fornite dal prof, nella cartella `/matrici/` della repository.

Gestione degli errori

Per avere una corretta implementazione degli errori, la libreria segue gli standard Java e clean coding.

Nella classe `main`, gli errori sono visualizzati sullo stream di output `System.err`, al posto di `System.out`, per una migliore differenza tra i messaggi generici e quelli di errore.

Le funzioni di supporto e i metodi iterativi, inoltre, fanno uso di eccezioni per un'efficace gestione degli errori. Ogni eccezione è di diverso tipo, a seconda dell'errore, avendo comunque un efficace messaggio che spiega all'utente cosa non sta funzionando. Ogni eccezione lanciata dal programma è gestita attraverso gli appositi blocchi `try-catch`.

Funzione main

La struttura del `main` è organizzata come segue:

- Inizializzazione costante del numero massimo di iterazioni
- Caricamento della matrice in formato `mtx`, nome del file passato come primo argomento
- Inizializzazione del vettore `x0` con i valori impostati a 0
- Inizializzazione del vettore `x` con i valori impostati a 1
- Inizializzazione del vettore `b = Ax`
- Esecuzione di tutti i metodi iterativi in base alla matrice passata e ai vettori appena creati

Package Funzioni

Alcune delle funzioni presenti in questo package servono ad evitare ripetizioni di codice nelle diverse classi del programma.

Il package Funzioni include le seguenti funzioni:

- `public static DMatrixSparseCSC caricaMatriceManualmente(String percorso):` caricamento matrice dal file `mtx` passato come parametro
- `public static boolean isNotDiagonaleZero(DMatrixSparseCSC A):` controllo dell'esistenza di zeri sulla diagonale della matrice sparsa passata come parametro
- `public static DMatrixSparseCSC triang_inf(final DMatrixSparseCSC A):` creazione della matrice triangolare inferiore a partire dalla matrice sparsa passata come parametro
- `public static DMatrixRMaj solveTriangInf(DMatrixSparseCSC L, DMatrixRMaj b):` risoluzione del sistema $Lx=b$ usando la formula forward-substitution

- `public static void ritornaValori(double tol, Risultato r):` stampa le soluzioni di uno dei metodi iterativi, passando come parametri la tolleranza e un oggetto Risultato
- `public static double calcoloNormaDiff(final DMatrixSparseCSC A, final DMatrixRMaj x0, final DMatrixRMaj b):` calcola la norma euclidea usando la formula $\frac{\|A \cdot x^{(k)} - b\|}{\|b\|}$
- `public static double calcoloErroreRelativo(int n, DMatrixRMaj xNew):` calcolo errore relativo della soluzione di un metodo rispetto alla soluzione esatta
- `public static void controlloDimensione(int n, int m, int L) throws Exception:` controlla che la matrice sia quadrata e che la dimensione del vettore x0 sia uguale alla matrice

Metodi iterativi

I metodi iterativi vengono svolti utilizzando come criterio di arresto la funzione `calcoloErroreRelativo()` precedentemente descritta. Tutti i metodi partono da una soluzione iniziale nulla ed è previsto un numero massimo di iterazioni (20000) specificato nel main del programma.

Tutti i metodi implementano una chiamata alla funzione `controlloDimensione()` precedentemente descritta, che controlla se la matrice passata è quadrata, se la dimensione del vettore soluzione è uguale a quella della matrice, e se la matrice è simmetrica.

Metodo di Jacobi

Il metodo di Jacobi calcola prima il vettore residuo globale $b - Ax^{(k)}$, che misura l'errore del sistema in quel momento, lo riscalda componente per componente tramite la diagonale inversa D^{-1} , e usa questo per aggiornare la soluzione.

$$x^{(k+1)} = x^{(k)} + D^{-1}(b - Ax^{(k)})$$

Oltre al controllo della dimensione comune a tutte le funzioni, si verifica anche che non siano presenti zeri sulla diagonale, per evitare una divisione per zero.

Metodo di Gauss-Seidel

Il metodo sfrutta le funzioni di supporto `triangInf()` e `solveTriangInf()`, discusse precedentemente, per risolvere il sistema. Considerato che sfrutta i valori già aggiornati nella stessa iterazione, questo metodo risulta, in generale, essere più veloce rispetto a Jacobi.

$$Lx^{(k+1)} = b - Bx^{(k)}$$

Come il metodo di Jacobi, si verifica che non siano presenti zeri sulla diagonale.

Metodo del gradiente

Ad ogni iterazione k , il metodo del gradiente si muove lungo la direzione $p(k) = r^{(k)} = b - Ax^{(k)}$.

Per calcolare il passo, si usa $\alpha_k = \frac{r^{(k)T}r^{(k)}}{r^{(k)T}Ar^{(k)}}$; al termine di ciò, viene aggiornata la soluzione con $x^{(k+1)} = x^{(k)} + \alpha_k r^{(k)}$.

Metodo del gradiente coniugato

Per ogni passo k , il metodo del gradiente coniugato:

- Calcola il passo $\alpha_k = \frac{r^{(k)T}r^{(k)}}{p^{(k)T}Ap^{(k)}}$
- Aggiorna la soluzione $x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$
- Aggiorna il residuo $r^{(k+1)} = r^{(k)} - \alpha_k Ap^{(k)}$
- Calcola il coefficiente di correzione $\beta_k = \frac{r^{(k+1)T}r^{(k+1)}}{r^{(k)T}r^{(k)}}$
- Imposta la nuova direzione coniugata come $p^{(k+1)} = r^{(k+1)} + \beta_k p^{(k)}$

Esecuzione del programma

Considerato che il progetto usa Maven per gestire le dipendenze, abbiamo ritenuto importante delineare come si esegue il programma.

Per la prima esecuzione, è importante eseguire nel terminale `mvn clean compile`, quando si è posizionati nella root della repository, per compilare il progetto ed assicurarsi che tutte le dipendenze siano caricate correttamente in Maven.

Il programma si aspetta due argomenti: `args[0]` (il nome del file della matrice), e `args[1]` (la tolleranza). Per eseguire il programma:

```
mvn exec:java -Dexec.args="percorso_relativo/nome_matrice.mtx 1e-6"
```

Una volta eseguito tale comando, il programma eseguirà tutti i metodi iterativi sulla matrice specificata, con la tolleranza specificata.

Risultati

Di seguito, vengono mostrati i risultati delle esecuzioni dei quattro metodi iterativi su quattro matrici sparse di esempio fornite: `spa1`, `spa2`, `vem1` e `vem2`.

Per ciascuna matrice è stato creato:

- Il vettore x_0 , di tutti zeri, per salvare le soluzioni finali dei metodi iterativi
- Il vettore x , di tutti uni, rappresentativo della soluzione finale. La soluzione esatta è nota a priori, e può essere usata per il calcolo dell'errore relativo.
- Il termine noto b , calcolato come $b = Ax$

Gli esperimenti sono stati condotti con quattro diversi valori della tolleranza di arresto: 10^{-4} , 10^{-6} , 10^{-8} , 10^{-10} . Per ogni metodo iterativo e per ogni matrice, sono stati registrati il numero di iterazioni necessarie per la convergenza, l'errore relativo finale rispetto alla soluzione, e il tempo di esecuzione.

spa1.mtx

Tolleranza	Jacobi			Gauss-Seidel			Gradiente			Gradiente coniugato		
	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.
10^{-4}	0.2887	0.002	115	0.2289	0.018	9	0.1598	0.034	143	0.0492	0.021	49
10^{-6}	0.4446	$1.79 \cdot 10^{-5}$	181	0.4212	$1.29 \cdot 10^{-4}$	17	3.3485	$9.68 \cdot 10^{-4}$	3577	0.1176	$2.55 \cdot 10^{-5}$	134
10^{-8}	0.6254	$1.82 \cdot 10^{-7}$	247	0.5338	$1.71 \cdot 10^{-6}$	24	6.4008	$9.81 \cdot 10^{-6}$	8233	0.1096	$1.32 \cdot 10^{-7}$	177
10^{-10}	0.6057	$1.85 \cdot 10^{-9}$	313	0.6530	$2.25 \cdot 10^{-8}$	31	9.0382	$9.82 \cdot 10^{-8}$	12919	0.1170	$1.20 \cdot 10^{-9}$	200

spa2.mtx

Tolleranza	Jacobi			Gauss-Seidel			Gradiente			Gradiente coniugato		
	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.
10^{-4}	0.5617	0.0017	36	2.2688	0.0025	5	1.2385	0.0181	161	0.3179	0.0098	42
10^{-6}	0.9849	$1.66 \cdot 10^{-7}$	57	3.6229	$5.14 \cdot 10^{-5}$	8	14.9333	$6.69 \cdot 10^{-4}$	1949	0.9179	$1.20 \cdot 10^{-4}$	122
10^{-8}	1.2892	$1.57 \cdot 10^{-7}$	78	5.6535	$2.79 \cdot 10^{-7}$	12	38.2157	$6.86 \cdot 10^{-6}$	5087	1.1620	$5.58 \cdot 10^{-7}$	196
10^{-10}	1.4708	$1.48 \cdot 10^{-9}$	99	6.1959	$5.57 \cdot 10^{-8}$	15	53.5178	$6.93 \cdot 10^{-8}$	8285	1.0920	$5.32 \cdot 10^{-9}$	240

vem1.mtx

Tolleranza	Jacobi			Gauss-Seidel			Gradiente			Gradiente coniugato		
	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.
10^{-4}	0.4233	0.003	1314	3.8536	0.003	659	0.1232	0.003	890	0.0034	$4.08 \cdot 10^{-5}$	38

10^{-6}	0.5673	$3.54 \cdot 10^{-5}$	2433	7.3486	$3.53 \cdot 10^{-5}$	1218	0.1748	$2.71 \cdot 10^{-5}$	1612	0.0035	$3.73 \cdot 10^{-7}$	45
10^{-8}	0.6137	$3.54 \cdot 10^{-7}$	3552	9.6935	$3.52 \cdot 10^{-7}$	1778	0.2181	$2.70 \cdot 10^{-7}$	2336	0.0035	$2.84 \cdot 10^{-9}$	53
10^{-10}	0.7498	$3.54 \cdot 10^{-9}$	4671	12.737	$3.51 \cdot 10^{-9}$	2338	0.3594	$2.71 \cdot 10^{-9}$	3058	0.0156	$2.19 \cdot 10^{-11}$	59

vem2.mtx

Tolleranza	Jacobi			Gauss-Seidel			Gradiente			Gradiente coniugato		
	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.	Tempo	Err. Rel.	n. iter.
10^{-4}	0.5433	0.005	1927	15.286	0.005	965	0.2215	0.0038	1308	0.0073	$5.72 \cdot 10^{-5}$	47
10^{-6}	1.1164	$4.97 \cdot 10^{-5}$	3676	26.855	$4.94 \cdot 10^{-5}$	1840	0.3237	$3.79 \cdot 10^{-5}$	2438	0.0062	$4.74 \cdot 10^{-7}$	56
10^{-8}	1.3084	$4.96 \cdot 10^{-7}$	5425	40.994	$4.96 \cdot 10^{-7}$	2714	0.3781	$3.81 \cdot 10^{-7}$	3566	0.0158	$4.30 \cdot 10^{-9}$	66
10^{-10}	1.7807	$4.96 \cdot 10^{-9}$	7174	52.307	$4.95 \cdot 10^{-9}$	3589	0.6048	$3.79 \cdot 10^{-9}$	4696	0.0093	$2.25 \cdot 10^{-11}$	74

Come si può notare dalle tabelle precedentemente scritte:

- Il metodo di Jacobi e di Gauss-Seidel risultano notevolmente più veloci con le matrici spa1 e spa2, che con le matrici vem1 e vem2
- Il metodo del Gradiente è decisamente più lento di tutti gli altri metodi in spa1 e spa2
- Il metodo di Gauss-Seidel è decisamente più lento di tutti gli altri metodi in vem1 e vem2,
- Il metodo del gradiente risulta notevolmente più veloce nelle matrici vem1 e vem2; a partire dalla tolleranza 10^{-6} , anche il numero di iterazioni è meno elevato rispetto ai test con le matrici spa1 e spa2, mentre con la tolleranza 10^{-4} , il numero di iterazioni è più elevato
 - o Si può notare lo stesso fenomeno anche con il gradiente coniugato, seppure in ordine minore
- L'errore relativo per i metodi Jacobi e Gauss-Seidel nelle matrici vem1 e vem2 si avvicina alla tolleranza richiesta, ma si attesta sempre su un ordine di grandezza superiore
- L'errore relativo per il gradiente coniugato è relativamente efficace nelle matrici vem1 e vem2, superando la precisione minima richiesta dal test di arresto

Il metodo del gradiente coniugato si dimostra essere il metodo migliore in ogni scenario testato.